

Week 3: Seller and Inventory Management System

Scenario:

The movie ticket booking company works with multiple cinemas that show different movies. The system should maintain cinema details, track the movies shown by each cinema, and monitor ticket availability to ensure tickets are available for customers.

1. Create Seller and Inventory Tables:

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7), Seller_Name VARCHAR(25), Email VARCHAR(30), Phone NUMBER(10), Address VARCHAR(45));  
Table created.
```

```
SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7), Product_ID VARCHAR(7), Seller_ID VARCHAR(7), Stock_Quantity NUMBER, Stock_Status VARCHAR(11), Last_Updated DATE);  
Table created.
```

Shows the CREATE TABLE commands used to create the Seller and Inventory tables.

2. Establish Relationships:

Seller and Product:

One seller can supply multiple products.

One product can be supplied by one or multiple sellers.

Relationship: Seller → Product is one to many

Product and Inventory:

Each product has inventory information.

Inventory tracks available stock.

Relationship: Product → Inventory is one to one

Primary Key added for Both Tables:

Shows the primary key constraints added to the Category and Product tables.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY (Seller_ID);  
Table altered.  
  
SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inventory PRIMARY KEY (Inventory_ID);  
Table altered.
```

Foreign Key for Inventory Table:

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);  
Table altered.  
  
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);  
Table altered.
```

Shows the foreign key constraint added to the Inventory table.

Constraints to make Unique Email ID:

Shows the unique constraint added to the Seller email column.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
Table altered.
```

To make Email NOT NULL:

Shows the NOT NULL constraint applied to the Seller email column.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
Table altered.
```

Add Constraint to Check the Stock Quantity is Positive:

Shows the CHECK constraint ensuring Stock_Quantity is greater than zero.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity>=0);  
Table altered.
```

Insert Values to Seller Table:

Shows seller records being inserted into the Seller table.

```
SQL> insert into seller values('S001','PVR Cinemas','pvr@gmail.com',9876543210,'Chennai');  
1 row created.  
SQL> insert into seller values('S002','INOX Cinemas','inox@gmail.com',9876543211,'Tambaram');  
1 row created.  
SQL> insert into seller values('S003','AGS Cinemas','ags@gmail.com',9876543202,'Chennai');  
1 row created.  
SQL> insert into seller values('S004','Rohini Cinemas','rohini@gmail.com',9876543212,'Koyambedu');  
1 row created.  
SQL> insert into seller values('S005','Luxe Cinemas','lux@gmail.com',9876543254,'Velachery');  
1 row created.  
SQL> insert into seller values('S006','Kamala Cinemas','kamala@gmail.com',9876543145,'Anna Nagar');  
1 row created.  
SQL> insert into seller values('S007','EGA Cinemas','ega@gmail.com',9876543190,'T Nagar');  
1 row created.  
SQL> insert into seller values('S008','Vetri Cinemas','vetri@gmail.com',9876432150,'Chrompet');  
1 row created.  
SQL> insert into seller values('S009','Sathyam Cinemas','satyam@gmail.com',9876432150,'Royapet');  
1 row created.  
SQL> insert into seller values('S010','SPI Cinemas','spi@gmail.com',9876432100,'Vadapalani');  
1 row created.
```

Shows the Seller table after inserting and displaying the seller records.

```
SQL> select*from seller;
```

SELLER_ID	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S001	PVR Cinemas	pvr@gmail.com	9876543210
Chennai			
S002	INOX Cinemas	inox@gmail.com	9876543211
Tambaram			
S003	AGS Cinemas	ags@gmail.com	9876543202
Chennai			
SELLER_ID	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S004	Rohini Cinemas	rohini@gmail.com	9876543212
Koyambedu			
S005	Luxe Cinemas	luxe@gmail.com	9876543254
Velachery			
S006	Kamala Cinemas	kamala@gmail.com	9876543145
Anna Nagar			
SELLER_ID	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S007	EGA Cinemas	ega@gmail.com	9876543190
T Nagar			

S008	Vetri Cinemas	vetri@gmail.com	9876432150
Chrompet			
S009	Sathyam Cinemas	satyam@gmail.com	9876432150
Royapet			
SELLER_ID	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S010	SPI Cinemas	spi@gmail.com	9876432100
Vadapalani			
10 rows selected.			

Insert Values to Inventory Table:

Shows inventory records being inserted into the Inventory table.

```
SQL> insert into inventory values('I001','P001','S001',50,'Available',to_date('15-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I002','P002','S002',40,'Available',to_date('17-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I003','P003','S003',35,'Available',to_date('19-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I004','P004','S004',45,'Available',to_date('20-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I005','P005','S005',30,'Available',to_date('21-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I006','P006','S006',50,'Available',to_date('22-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I007','P007','S007',25,'Low Stock',to_date('23-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I008','P008','S008',40,'Available',to_date('24-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I009','P009','S009',20,'Low Stock',to_date('25-09-2026','DD-MM-YY'));
1 row created.

SQL> insert into inventory values('I010','P010','S010',10,'Low Stock',to_date('26-09-2026','DD-MM-YY'));
1 row created.
```

Shows the Inventory table records after insertion.

```
SQL> select*from inventory;

INVENTO PRODUCT_ID  SELLER_ID  STOCK_QUANTITY  STOCK_STATU  LAST_UPDA
-----
I001      P001          S001              50 Available    15-SEP-26
I002      P002          S002              40 Available    17-SEP-26
I003      P003          S003              35 Available    19-SEP-26
I004      P004          S004              45 Available    20-SEP-26
I005      P005          S005              30 Available    21-SEP-26
I006      P006          S006              50 Available    22-SEP-26
I007      P007          S007              25 Low Stock    23-SEP-26
I008      P008          S008              40 Available    24-SEP-26
I009      P009          S009              20 Low Stock    25-SEP-26
I010      P010          S010              10 Low Stock    26-SEP-26

10 rows selected.
```

Report 1: Seller-wise Product Report

Display:

- Seller name
- Number of products supplied
- Available stock

Shows the Seller-wise Product Report with seller name, product count, and available stock.

```
SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
PVR Cinemas	1	50
Luxe Cinemas	1	30
AGS Cinemas	1	35
INOX Cinemas	1	40
Rohini Cinemas	1	45
Kamala Cinemas	1	50
Vetri Cinemas	1	40

7 rows selected.

Use Joins to get details from Seller and Inventory Table.

Report 2: Stock Availability Report

Display:

- Product name
- Stock quantity
- Stock status

Shows the Stock Availability Report with product name, stock quantity, and stock status.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATU
Leo	50	Available
Good Night	40	Available
96	35	Available
Love Today	45	Available
Conjuring	30	Available
Vikram	50	Available
Interstellar	25	Low Stock
Toy Story	40	Available
Avengers	20	Low Stock
The Lion King	10	Low Stock

10 rows selected.

Report 3: Inventory Analysis:

Find:

- Total products available

Shows the total number of products currently available.

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_Status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
7

- **Products out of stock:**

Shows the number of products that are out of stock.

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Status = 'Out of Stock';

OUTOFSTOCK_PRODUCTS
-----
0
```

- **Highest stocked products:**

Shows the product(s) with the highest stock quantity.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);

PRODUCT_NAME      STOCK_QUANTITY
-----
Leo                50
Vikram             50
```

- **Average inventory quantity:**

Shows the average inventory quantity calculated using AVG(Stock_Quantity).

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;

AVERAGE_INVENTORY
-----
34.5
```